Львівський національний університет імені Івана Франка · Факультет електроніки та комп'ютерних технологій · Кафедра системного проєктування

Вибрані розділи науки про дані · 2026–2027 н. р.

Лабораторна робота 14

Розгортання та моніторинг дрейфу

- **Тема:** ЛР 14. Розгортання та моніторинг дрейфу
- **Лекція:** 14-й тиждень, лекція 14 «Проєктування систем, керованих даними: MLOps, DataOps та AutoML»
- **Тривалість:** 2 год аудиторно + ≈ 4 год самостійної підготовки й оформлення
- **Оцінювання:** максимум 2 бали = 5 результатів × 0.3 бала (разом 1.5) + 0.5 бала за захист (5 запитань × 0.1)
- **Термін здачі та захисту:** до кінця 15-го тижня
- **Інструменти:** Docker, Evidently, MLflow
- **Дані / вхід:** система з ЛР 12; еталонний (reference) розподіл ознак/скорів із навчальних або історичних запитів

1. Мета роботи

Підготувати систему до експлуатації: контейнеризувати, додати журнали й метрики, реалізувати виявлення дрейфу (PSI, KS) та описати план реагування.

2. Які навички ви наберете

- пакувати застосунок у Docker і описувати середовище;
- налаштовувати журнали та метрики експлуатації;
- вимірювати дрейф даних і налаштовувати пороги тривог;
- складати runbook для інцидентів і планувати безпечне розгортання.

3. Підготовка

- Опрацюйте лекцію 14 та джерела [14, 20, 21].
- Встановіть Docker Desktop; за бажанням MLflow, Evidently.
- Підготуйте reference-вибірку (≥ 1000 значень) для 3 ознак або скорів.

4. Порядок виконання

Робота складається з п'яти результатів. Кожен результат — самостійна частина (таблиця, схема, алгоритм, модель, код, метрики тощо) і оцінюється окремо: 0.3 бала — виконано повністю; 0.15 — виконано частково; 0 — не виконано. Виконуйте результати послідовно: кожен наступний використовує попередній.

Результат 1. Контейнеризація

Крок 1. Складіть requirements.txt із зафіксованими версіями.

Крок 2. Напишіть Dockerfile (базовий образ python:3.11-slim, непривілейований користувач, EXPOSE, CMD uvicorn).

Крок 3. Складіть docker-compose.yml: API + Neo4j (+ індекс/том для даних), змінні середовища через .env.

Крок 4. Додайте healthcheck і перевірте запуск однією командою docker compose up.

Що зараховується. Повністю (0.3 бала): запуск однією командою, версії зафіксовано, healthcheck, секрети поза образом. Частково (0.15 бала): запуск потребує ручних кроків. Інакше — 0.

Результат 2. Журнали та метрики експлуатації

Крок 1. Реалізуйте структуровані журнали (JSON): час, id запиту, версія моделі й підказки, затримка, токени.

Крок 2. Додайте лічильники й гістограми метрик (prometheus_client або власні): запити, помилки, затримка, вартість.

Крок 3. Створіть ендпоінт /metrics або файл метрик.

Крок 4. Виконайте 50 тестових запитів і покажіть зведення метрик.

Що зараховується. Повністю (0.3 бала): журнали + метрики + версії, зведення за 50 запитів. Частково (0.15 бала): лише текстові журнали без метрик. Інакше — 0.

Результат 3. Виявлення дрейфу

Крок 1. Реалізуйте функції psi(expected, actual, bins=10) і KS-тест (scipy.stats.ks_2samp).

Крок 2. Змоделюйте два види змін: зсув середнього на 1σ та зміну формату/одиниць ознаки (×100).

Крок 3. Для кожного випадку обчисліть PSI та p-значення KS для 3 ознак/скорів.

Крок 4. Побудуйте графіки розподілів «еталон vs поточний» і PSI у часі (по тижнях).

Крок 5. Визначте пороги (напр. PSI 0.1/0.25) і обґрунтуйте їх.

Що зараховується. Повністю (0.3 бала): PSI і KS, два типи зсувів, графіки, обґрунтовані пороги. Частково (0.15 бала): один метод або без модельованих зсувів. Інакше — 0.

Результат 4. Тривоги та реагування

Крок 1. Реалізуйте перевірку порогів, що створює тривогу рівня warning/critical.

Крок 2. Налаштуйте придушення повторних тривог (cooldown) і підтвердження за двома ознаками.

Крок 3. Змоделюйте 3 сценарії (дрейф даних, дефект даних, відмова LLM) і покажіть, які тривоги спрацювали.

Крок 4. Порахуйте хибні тривоги на «спокійних» даних і запишіть, як їх зменшити.

Що зараховується. Повністю (0.3 бала): рівні тривог, придушення, 3 сценарії, оцінка хибних тривог. Частково (0.15 бала): тривоги без сценаріїв або без оцінки хибних. Інакше — 0.

Результат 5. Runbook і план розгортання

Крок 1. Складіть runbook: тривога → класифікація → дія (виправити джерело/відкат/перенавчання) з відповідальними й часом реакції.

Крок 2. Опишіть критерії відкату та метрики успіху для нової версії.

Крок 3. Оберіть шаблон розгортання (shadow, canary, blue-green) і обґрунтуйте вибір для вашої системи.

Крок 4. Запишіть вимоги до data contract з постачальником даних.

Крок 5. Заповніть README і декларацію ШІ.

Що зараховується. Повністю (0.3 бала): runbook із відповідальними, критерії відкату, обґрунтований шаблон розгортання, data contract. Частково (0.15 бала): без критеріїв відкату або відповідальних. Інакше — 0.

5. Вимоги до звіту та файлів

- Звіт за шаблоном «Шаблон звіту до лабораторної роботи»; для кожного результату — розділ із описом кроків, фрагментами коду, таблицями, рисунками та висновком.
- Код (ноутбук/скрипти) запускається з нуля; є README (версії, залежності, фіксований seed, інструкція запуску).
- Файли: ЛР14_{Прізвище}_{група}.pdf та .zip; ключі API, паролі, персональні дані не включати.
- Здача в Moodle до терміну; захист — усно на лабораторному занятті.

6. Оцінювання

- **Результати 1–5:** по 0.3 бала (максимум 1.5): повністю — 0.3; частково — 0.15; не виконано — 0. Умови наведено під кожним результатом.
- **Захист і опитування:** 0.5 бала (5 запитань по 0.1 бала); студент також має пояснити будь-яку частину роботи й за вимогою змінити її «наживо».
- **Запізнення** (до закриття модуля): бали за результати множаться на 0.8; після закриття модуля робота не приймається, крім поважних причин.
- **Не пояснив роботу:** бали за результати зменшуються вдвічі (до кратних 0.15).
- **Порушення доброчесності** (списування, вигадані дані чи джерела, неоголошене використання ШІ): робота не зараховується.
- **Декларація використання ШІ** обов’язкова (інструмент, призначення, запити, як перевірено); без неї результати не зараховуються.

7. Контрольні питання (захист)

1. Чим дрейф даних відрізняється від дрейфу концепції?
2. Як інтерпретувати PSI = 0.3?
3. Що таке shadow, canary та blue-green?
4. Яку роль має реєстр моделей?
5. Чому дефект даних може виглядати як дрейф?

8. Академічна доброчесність і використання ШІ

Робота виконується самостійно. Генеративний ШІ дозволено як допоміжний інструмент за умови декларації й розуміння кожної частини роботи. Списування, вигадані дані чи джерела, неоголошене використання ШІ — порушення доброчесності.

9. Рекомендовані джерела

- [14] Huyen; [20] Gama et al.; [21] Sculley et al.; документація Docker: https://docs.docker.com; Evidently: https://www.evidentlyai.com